

Three Benchmarks, Three Winners

A Measured Comparison of the Predator Provers
Premise Selection, Search Policy, and the Cost of a Node

Brian Tenneson
M.S., Applied Data Science
btenneson2301@baypath.edu

July 28, 2026

Abstract

Seven systems have been built under the name Predator. They rank different objects, over different corpora, in different units, and for most pairs the question “which is better” is not well posed: a premise selector’s *fraction of the library read* and a search policy’s *nodes expanded* are not commensurable quantities, and reporting them in one column would be the easiest available way to mislead a reader.

Partitioned into three benchmarks on which comparison *is* well posed, the systems separate cleanly. Over set.mm, Predator_4 wins premise selection on every metric and its performance is flat from four thousand statements to fifty thousand. Over the condensed-detachment fragment, Predator_5 expands $2.14\times$ fewer nodes than breadth-first search and—this is the claim that had never been checked—finishes $1.76\times$ faster in wall clock, while solving every target and returning a shortest proof 99% of the time.

Two results cut across the benchmarks. A linear ranker beat a random forest in three independent settings, which is evidence about the features rather than about the forest. And every search strategy that *discards* states—beam search, top- k pruning—finished slower in seconds than unguided breadth-first search *and* solved fewer targets. Proof-covering theory predicts that discarding forfeits a completeness theorem; these measurements say it also forfeits speed, so on this fragment the guarantee was free.

A general rule falls out of the timing data, and it is a statement about domains rather than about policies: a learned policy pays only when expanding a node is expensive relative to scoring one. Predator_1’s beam search lost $2.5\times$ in wall clock while opening $18\times$ fewer nodes, not because its policy was poor but because its nodes cost $21\mu s$ and were not worth avoiding.

Contents

1	What can be compared	2
2	Theory: what a policy may and may not do	2
3	Theory: when a policy is worth its cost	3
4	League A: premise selection over set.mm	3
5	League B: search policy over the fragment	3
6	League C: strategies timed in seconds	4

7	Why the same architecture won here and lost before	5
8	Limitations	5
9	Conclusion	6

1 What can be compared

System	Ranks	Corpus	Metric
Predator_1	proof-graph nodes	synthetic, 789	expansions, seconds
Predator_2	lemmas	set.mm, 8k	recall@ k , effort
Predator_3	lemmas	set.mm, 2k–32k	recall@ k , effort
Predator_4	lemmas	set.mm, 50k	recall@ k , effort
python_rf	lemmas	set.mm, 16k	recall@ k , effort
Predator_5	edges	CD fragment	expansions vs. BFS
Predator_6	edges	CD fragment	expansions vs. BFS

Predator_4’s $3.5\times$ is a fraction of the library that must be read. Predator_5’s $1.8\times$ is nodes expanded before a target appears. These are different quantities and there is no exchange rate between them. The systems are therefore judged in three leagues: premise selection over set.mm, search policy over the fragment, and search strategy timed in seconds.

2 Theory: what a policy may and may not do

Definition 2.1 (Proof-covering). A nondeterministic proof-search policy Σ is *proof-covering* if for every finite proof $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ from Γ there is a branch $b = \langle p_1, p_2, \dots \rangle$ of the search with $\{\pi(1), \dots, \pi(j)\} \subseteq p_j$ for every $j \leq n$.

Under that hypothesis the Branch-Covering Theorem applies: if $\Gamma \vdash_F \varphi$ then some target-search branch reaches φ by stage $\|\varphi\|_F(\Gamma)$. Two ways of consuming a learned score have different standing with respect to it.

Proposition 2.2 (Reordering preserves proof-covering). *Let Σ_{full} return every admissible one-step extension at p , and let Σ_θ return the same set permuted by a scoring function θ . If Σ_{full} is proof-covering then so is Σ_θ .*

Proof. A branch is determined by which extensions are *present* in each $\Sigma(p_m, m)$, not by the order in which they are listed. Since $\Sigma_\theta(p, m) = \Sigma_{\text{full}}(p, m)$ as sets, the two policies generate the same set of branches, so any branch shadowing π under one shadows it under the other. $\square$

Proposition 2.3 (Truncation destroys it). *Let $\Sigma_{\theta, k}$ return only the top k extensions by θ . Then $\Sigma_{\theta, k}$ need not be proof-covering.*

Proof. It suffices that some state have its unique proof-continuing extension scored outside the top k . Nothing ties a fitted θ to geodesic membership, and §6 reports the failure occurring: pruned and beam policies lost targets that every complete policy solved. $\square$

Beam search is subject to Proposition 2.3 for the same reason: retaining the best W states per level discards the rest permanently.

3 Theory: when a policy is worth its cost

Node counts are hardware-independent, which is exactly why they can mislead—a node count does not charge for deciding. Let E denote expansions and c cost per expansion.

Definition 3.1. The *node speedup* is $\sigma = E_{\text{BFS}}/E_\pi$ and the *per-node overhead* is $\omega = c_\pi/c_{\text{BFS}}$.

Proposition 3.2 (Break-even). *The wall-clock speedup of policy π over breadth-first search is σ/ω . In particular π is faster in real time if and only if $\sigma > \omega$.*

Proof. Total time is expansions times cost per expansion, so

$$\frac{T_{\text{BFS}}}{T_\pi} = \frac{E_{\text{BFS}} c_{\text{BFS}}}{E_\pi c_\pi} = \frac{\sigma}{\omega}. \quad \square$$

Trivial, and it is the proposition the project spent the longest not applying. Predator_1 reported $\sigma = 18.1$ with $\omega = 41$, hence $\sigma/\omega = 0.44$: eighteen times fewer nodes and two and a half times slower.

4 League A: premise selection over set.mm

System	Corpus	r@10	r@50	MRR	Effort	vs. brute
Predator_2 (early)	8k	0.160	0.333	0.436	0.3513	2.6×
Predator_2 (later)	8k	0.232	0.491	0.479	0.3871	2.4×
Predator_2 (poor features)	8k	0.059	0.147	0.159	0.7470	1.2×
Predator_3	32k	0.275	0.600	0.668	0.6417	1.5×
Predator_4	50k	0.419	0.730	0.885	0.2866	3.5×
python_rf (forest)	16k	0.163	0.497	0.567	0.7007	1.4×

Finding 1. Predator_4 wins League A on every metric, at the largest corpus.

Finding 2 (Scaling). Recall@10 as the corpus grows from 2k to 32k statements:

Predator_3: 0.326, 0.253, 0.143, 0.228, 0.275

Predator_4: 0.336, 0.412, 0.413, 0.431, 0.414

Predator_3 swings by a factor of 2.3 with no mechanism to explain it and is measuring variance. Predator_4 is flat to ± 0.02 from 4k onward, and a flat curve is itself the result: performance holds as the library grows.

Finding 3 (Features dominate models). The same Predator_2 code scored recall@10 of 0.059 and 0.232 depending only on which features were active—a fourfold swing, larger than any gap between versions. In the poor run the weight on local co-citation was -0.94 ; in the good run, $+0.586$. One sign flip cost more than three versions of progress.

5 League B: search policy over the fragment

The fragment is condensed detachment over implicational formulas: one rule, most-general unification with occurs check, $\Gamma = \{K, S, W\}$. Its defining virtue is that $\Sigma(p, m)$ is finite and enumerable, which is what permits breadth-first search to compute true geodesics and hence to supply labels that are correct by construction rather than read from human-written proofs.

System	Depth 4	Depth 5
BFS ($\lambda = 0$)	25.6 exp.	75.5 exp.
Predator_5	15.5 $\pm$ 2.6 (1.60 $\times$)	42.3 $\pm$ 8.6 (1.78 $\times$)
Predator_6 ($p = 0.7$)	indistinguishable	indistinguishable
Predator_6.1 ($p = 0.9$)	indistinguishable	—

Finding 4 (Expert iteration is null here). Over a $p \times n$ grid paired on identical seeds, no setting beats $n = 1$. Predator_6.1 came in 0.5 expansions faster against a seed-to-seed standard deviation of 2.6, and Predator_6 costs 30–139% more expansions to reach the same place.

The mechanism is clear and it is not a defect in the implementation. Enlarging the bootstrap frontier creates nothing past the breadth-first horizon, and on this fragment breadth-first search prices *every* target. Certified labels are already geodesics; bootstrapped labels are upper bounds, and adding them to a training set that did not need them lowered the certified share to 22% at depth 5 and 10% at depth 6.

Remark 5.1 (A retracted hypothesis). An earlier draft claimed expert iteration pays wherever pass one leaves frontier targets unsolved, inferred from a single depth-5 versus depth-6 comparison. A controlled sweep refutes it. Depth 6 left 35% of its frontier unsolved—the most favourable case the hypothesis admits—and Predator_6 lost there, solving 22 against 24. Depth 4 left 0% and Predator_6 nominally won. The correlation runs backwards, and all three differences lay inside the seed noise in any case.

6 League C: strategies timed in seconds

Depth 5, 30 held-out targets, mean over 5 seeds. One ranker, one split; only the search strategy varies.

Strategy	Solved	Exp.	Sec.	$\mu s/exp$	Opt.	Guarantee
BFS ($\lambda = 0$)	100%	90.0	2.92	1083	100%	geodesic, complete
A* reorder	100%	42.0	1.66	1328	99%	complete
A* prune $k=4$	91%	77.8	3.48	1487	91%	incomplete
Beam $W=8$	70%	114.5	19.09	5483	80%	incomplete
Beam $W=1$	35%	29.4	4.02	4564	41%	incomplete

Applying Proposition 3.2:

Strategy	σ	ω	σ/ω
A* reorder	2.14	1.2	1.76
A* prune $k=4$	1.16	1.4	0.84
Beam $W=8$	0.79	5.1	0.15
Beam $W=1$	3.06	4.2	0.73

Finding 5 (The node advantage survives timing). Predator_5 achieves $\sigma = 2.14$ at $\omega = 1.2$, hence $\sigma/\omega = 1.76$: it is faster in seconds, not merely in nodes, while solving every target and returning a geodesic 99% of the time.

Finding 6 (Beam search is the weakest strategy measured). Beam search—Predator_1’s strategy—solved 70% of targets, expanded *more* nodes than unguided breadth-first search (114.5 against 90.0), and finished 6.6 $\times$ slower in seconds. Its $\omega = 5.1$ is structural: a level-synchronous beam scores every child of every state in a level, sorts them, and discards most, paying for scoring it never uses.

Finding 7 (Discarding costs more than a theorem). All three strategies that discard states finished slower than breadth-first search *and* solved fewer targets. The only strategy that merely reorders won on every axis. Proposition 2.3 says discarding forfeits the Branch-Covering Theorem; these measurements say it forfeits time and solve rate as well. On this fragment there was no speed-for-guarantee trade to make.

Finding 8 (Linear beats forest, three times). python_rf on set.mm: effort $0.2898 \rightarrow 0.7007$ at $6.4\times$ the fit time. Predator_3’s forest variant: recall@10 0.185, 0.063, 0.077. Predator_5’s ranker: 47.0 against 34.9 expansions. Three independent settings agree, which makes this evidence that the signal is close to linear *in these features*—not evidence that forests are unsuitable.

7 Why the same architecture won here and lost before

Predator_1 opened $18\times$ fewer nodes and lost $2.5\times$ in wall clock; Predator_5 opens $2.14\times$ fewer and wins $1.76\times$. The smaller node advantage produced the better outcome. The explanation is in ω , and it is a statement about the domain.

Breadth-first search costs $1083\ \mu s$ per expansion on the fragment; Predator_1’s brute force costs $21\text{--}28\ \mu s$. The difference is not the logic—both systems close forward from three axioms—but *when the graph is built*.

Predator_1 constructs its corpus once, by brute-force closure, reporting stage sizes $|D_F^{(m-1)}(G)| = [3, 39, 111, 381, 579, 789]$. Search then runs over a *precomputed* graph, so expanding a node is an adjacency lookup. Scoring it costs more than visiting it, and $\omega = 41$.

Predator_5 computes $\Sigma(p, m)$ *on demand*, attempting $D(x, y)$ for every ordered pair in the state with unification and an occurs check at every expansion. Visiting a node costs far more than scoring it, and $\omega = 1.2$.

Corollary 7.1. *A learned policy is worth its cost precisely when expanding a node is expensive relative to scoring one—equivalently, when the search graph is generated on demand rather than precomputed.*

Predator_1’s policy may have been perfectly good; its nodes were too cheap to be worth avoiding. The consequence for set.mm is immediate, and it now depends on an architectural choice rather than on model quality. A prover that searches the recorded citation DAG has precomputed edges, hence Predator_1’s regime, hence large ω . A prover that re-derives admissible extensions by unification has Predator_5’s regime and small ω —but that is the enumeration problem that drove Predator_2 to premise selection in the first place.

The two regimes are therefore not freely choosable. Cheap nodes come with a precomputed graph, which means the proofs are already known; expensive nodes come with genuine search. This is the central tension of the project stated in units of microseconds.

8 Limitations

The fragment is condensed detachment, not ZFC. Predator_5’s features are stated in its terms—unifier growth, membership in Γ —and there is no $\in$ or $\forall$ for them to describe, so League B and C results do not transfer to set.mm as stated.

Held-out sets are small: 18 to 39 targets, with seed-to-seed standard deviations of 2.6 to 11.7 expansions. Every figure quoted here is a mean over five or ten seeds for that reason, and single-run figures from earlier drafts of this work were not reliable.

League A and League B measure different things and their winners do not compete. Predator_4 cannot prove anything: it emits an ordering and stops. Predator_5 produces proofs but only within a 287-state fragment.

9 Conclusion

Three winners, one per benchmark. Predator_4 for premise selection over set.mm, on every metric, with performance flat to fifty thousand statements. Predator_5 for search policy, now the best-evidenced system in the project: fewer nodes, fewer seconds, complete, and geodesic 99% of the time. Logistic regression over random forests, in three independent settings.

One loser worth naming: every strategy that discards states. The theory predicted the completeness cost; the measurements added that there was no compensating speed. Where reordering suffices, the guarantee is free.

And one rule that outlives the numbers. Whether a learned policy pays is decided by the ratio of node cost to scoring cost, not by the quality of the policy. That ratio is a property of the domain, it is measurable before any model is fitted, and it should be measured first.